Модификация данных и программирование в SQL. Пример проектирования БД

старший преподаватель кафедры АСУ
Валова Анастасия Александровна

9 Модификация данных

Вставка данных в таблицы

- INSERT...VALUES
  - Добавляет явно указанные значения.
  - Можно опустить значения в столбцах identity, допускающих NULL столбцах и столбцах с умалчиваемыми значениями.
  - Можно явно указать NULL и DEFAULT.
- INSERT...SELECT / INSERT...EXEC
  - Вставка в существующую таблицу результатов выполнения другого запроса или хранимой процедуры
- SELECT...INTO
  - Создает новую таблицу на основании результатов запроса
  - Не поддерживается в Azure SQL Database

Создание идентификаторов

Использование идентифицирующих столбцов

- Свойство столбца IDENTITY создает автоматически последовательность чисел для вставки в таблицу
  - Можно задавать начальное значение и приращение
  - Системные функции и переменные для определения вставленного значения:
    - ✓ **@@IDENTITY**: Последнее значение в сессии
    - ✓ **SCOPE_IDENTITY()**: Последнее значение в сессии и области видимости
    - ✓ **IDENT_CURRENT('*<имя_таблицы>*')**: Последнее значение в таблице

```
INSERT INTO Sales.Orders (CustomerID, Amount)
VALUES
(12, 2056.99);
…
SELECT SCOPE_IDENTITY() AS OrderID;
```

Создание идентификаторов
Использование последовательностей

- Последовательности – объекты, генерирующие последовательные числа
  - Поддержка в SQL Server 2012 и позднее
  - Существуют независимо от таблиц, поэтому гибче, чем identity
  - Используйте SELECT NEXT VALUE FOR для извлечения следующего числа в последовательности
    - Можно установить в качестве значения по умолчанию для столбца

```
CREATE SEQUENCE Sales.OrderNumbers AS INT
START WITH 1 INCREMENT BY 1;
…
SELECT NEXT VALUE FOR Sales.OrderNumbers;
```

Обновление данных в таблице

Оператор UPDATE

- Обновляет все строки в таблице или представлении
  - Фильтрация строк для установки значений через предложение WHERE
  - Устанавливаемые значения можно определять из предложения FROM
- Модифицируются только столбцы, указанные в предложении SET

```
UPDATE Production.Product
SET unitprice = (unitprice * 1.04)
WHERE categoryid =  1 AND discontinued = 0;
```

Обновление данных в таблице

Оператор MERGE

MERGE модифицирует данные на основе условия

- Когда источник соответствует цели
- Когда источник не имеет соответствия с целью
- Когда цель не имеет соответствия с источником

```
MERGE INTO Production.Products as P                                    -- цель
        USING Production.ProductsStaging as S ON P.ProductID=S.ProductID    -- источник
WHEN MATCHED THEN
        UPDATE SET
        P.UnitPrice = S.UnitPrice, P.Discontinued=S.Discontinued
WHEN NOT MATCHED [BY TARGET] THEN
        INSERT (ProductName, CategoryID, UnitPrice, Discontinued)
        VALUES (S.ProductName, S.CategoryID, S.UnitPrice, S.Discontinued)
WHEN NOT MATCHED BY SOURCE THEN
        DELETE;
```

Удаление данных из таблицы

- DELETE без предложения WHERE удаляет все строки
  - Используйте предложение WHERE для отбора удаляемых строк

```
DELETE FROM SalesLT.SalesOrderDetail
WHERE SalesOrderID = 10248;
```

- TRUNCATE TABLE очищает всю таблицу
  - Хранилище физически очищается, строки не удаляются по отдельности
  - Минимальное журналирование (сбрасывает identity)
  - Можно откатить, если TRUNCATE выполняется в транзакции
  - TRUNCATE TABLE потерпит неудачу, если на таблицу указывает внешний ключ из другой таблицы

10 Процедурное программирование на Transact-SQL

Переменные

- Переменные – это объекты, которые позволяют сохранять значение для последующего использования в том же пакете
- Переменные объявляются с использованием оператора DECLARE
  - Переменные могут быть объявлены и инициализированы в том же операторе
- Переменные всегда локальны для пакета в котором они объявлены и выходят за область видимости при завершении пакета

```
-- Объявление и инициализация переменных
DECLARE @color nvarchar(15)='Black', @size nvarchar(5)='L';
-- Использование переменных в операторах SQL
SELECT *
  FROM Production.Product
  WHERE Color=@color and Size=@size;
GO
```

Условное ветвление

- IF...ELSE использует предикат для определения пути выполнения кода
  - Код в блоке IF исполняется, если предикат разрешается в TRUE
  - Код в блоке ELSE исполняется, если предикат разрешается в FALSE или UNKNOWN

```
IF @color IS NULL
    SELECT * FROM Production.Product;
ELSE
    SELECT * FROM Production.Product
    WHERE Color = @color;
```

```
IF @color IS NULL
begin
    SELECT * FROM Production.Product;
end
ELSE
begin
    SELECT * FROM Production.Product
    WHERE Color = @color;
end
```

Циклы

- WHILE позволяет выполнять код в цикле
- Операторы в блоке WHILE повторяются, пока предикат вычисляется в TRUE
- Цикл заканчивается, когда предикат вычисляется в FALSE или UNKNOWN
- Выполнение можно изменить через BREAK или CONTINUE

```
DECLARE @custid AS INT=1, @lname AS NVARCHAR(20);
WHILE @custid <=5
   BEGIN
      SELECT @lname = lastname FROM Sales.Customer
      WHERE customerid = @custid;
      PRINT @lname;
      SET @custid += 1;
   END;
```

Хранимые процедуры. Типы

- Объекты БД, инкапсулирующие выполняемый на сервере код
- Системные хранимые процедуры:
  - Имя начинается с **sp_**
  - Созданы в БД master
  - Предназначены для применения в любой БД
  - Часто используются системными администраторами
- Пользовательские хранимые процедуры :
  - Определены в локальной БД
  - Имя часто начинается с префикса **pr_**

Хранимые процедуры (Stored Procedures)

- Могут быть параметризованы (@param as *datatype* = *default* [OUT])
  - Входные параметры
  - Output parameters

```
CREATE PROCEDURE SalesLT.GetProductsByCategory (@CategoryID INT = NULL)
AS
IF @CategoryID IS NULL
      SELECT ProductID, Name, Color, Size, ListPrice
      FROM SalesLT.Product
ELSE
      SELECT ProductID, Name, Color, Size, ListPrice
      FROM SalesLT.Product
      WHERE ProductCategoryID = @CategoryID;
```

- Учитывается порядок параметров

Подсказка: Размещайте параметры с умалчиваемыми значениями в конце списка параметров (для гибкости)

Вызов хранимых процедур с параметрами

```
CREATE PROCEDURE dbo.pr_GetTopProducts
(@StartID as int = 1, @EndID as int = 20) AS ...
```

- По имени:
  - EXEC pr_GetTopProducts
  - @StartID = 1, @EndID = 10
- По позиции:
  - EXEC pr_GetTopProducts 1, 10

- Использование умалчиваемых значений
  - EXEC pr_GetTopProducts @EndID=10

Введение в PostgreSQL для пользователей SQL Server

DDL-команды

CREATE, ALTER, DROP, TRUNCATE, RENAME

```
CREATE TABLE table_name (
    column1 datatype1,
    column2 datatype2,
    ...
);
```

```
CREATE VIEW view_name AS
SELECT column1, column2, ...
FROM table_name
WHERE condition;
```

```
ALTER TABLE table_name ADD
COLUMN column_name datatype;
```

```
ALTER TABLE table_name ALTER
COLUMN column_name TYPE
new_datatype;
```

```
ALTER TABLE table_name ADD
CONSTRAINT constraint_name
CHECK (condition);
```

```
DROP TABLE table_name;
```

```
DROP VIEW view_name;
```

```
TRUNCATE TABLE table_name;
```

```
ALTER TABLE table_name RENAME
COLUMN column_name TO
new_column_name;
```

Свойство IDENTITY (MSSQL) и тип данных SERIAL (PostgreSQL)

SQL Server (IDENTITY):

- Используется для создания столбца, значения которого автоматически увеличиваются при вставке новых строк.
- Вы можете задать начальное значение и шаг увеличения.

PostgreSQL (SERIAL):

- Это псевдотип, который автоматически создает последовательность и создает столбец, который будет получать значения из этой последовательности.
- SERIAL не является реальным типом данных, а скорее удобным способом для создания последовательности

Начиная с PostgreSQL 10, рекомендуемый подход к созданию автоматически увеличивающихся столбцов заключается в использовании свойстве столбца "IDENTITY", что соответствует стандарту SQL и обеспечивает более согласованное поведение в различных системах баз данных.

Свойство IDENTITY (MSSQL) и тип данных SERIAL (PostgreSQL)

- Синтаксис

```
CREATE TABLE example_table (
        id INT IDENTITY(1,1) PRIMARY KEY,
        name VARCHAR(50),
        address VARCHAR(50)
);
```

```
CREATE TABLE example_table (
        id SERIAL PRIMARY KEY,
        name VARCHAR(50),
        address VARCHAR(50)
);
```

- Изменение значений

```
SET IDENTITY_INSERT Employees ON;
INSERT INTO Employees (Id, Name)
VALUES (10, 'John Doe');
SET IDENTITY_INSERT Employees OFF;
```

```
INSERT INTO Employees (Id, Name)
VALUES (10, 'John Doe');
```

Объектно-ориентированная концепция

Наследование таблиц

Недостатки:

- Сложность и обслуживание
- Ограниченная гибкость
- Производительность запросов
- Целостность данных
- Изменения схемы
- Поддержка инструментов

```
CREATE TABLE vehicles (
    vehicle_id SERIAL PRIMARY KEY,
    make VARCHAR(50) NOT NULL,
    model VARCHAR(50) NOT NULL,
    year INTEGER NOT NULL
);
```

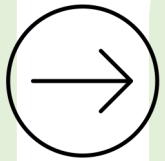


```
CREATE TABLE cars (
    color VARCHAR(20) NOT NULL,
    num_doors INTEGER NOT NULL,
    FOREIGN KEY (vehicle_id) REFERENCES vehicles
(vehicle_id)
) INHERITS (vehicles);
```

Объектно-ориентированная концепция

Пользовательские типы данных (UDT)

```
CREATE TYPE point AS (
    x FLOAT,
    y FLOAT
);
```



```
CREATE TABLE shapes (
    shape_id SERIAL PRIMARY KEY,
    shape_name VARCHAR(100) NOT NULL,
    position point
);

INSERT INTO shapes (shape_name, position)
VALUES
    ('Circle', ROW(2.5, 3.8)),
    ('Square', ROW(1.0, 1.0));
```



```
SELECT shape_name, position.x, position.y  FROM shapes;
```

DML-команды

UPSERT(Uplifting Database Updates and Inserts)

Преимущества:

- Упрощенная логика
- Улучшенная производительность
- Атомарность
- Управление параллелизмом

Недостатки:

- Сложность
- Блокировка и влияние на производительность
- Ограниченная совместимость

```
INSERT INTO table_name (column1, column2, ...)
VALUES (value1, value2, ...)
ON CONFLICT (constraint_columns)
DO UPDATE SET column1 = value1, column2 =
value2, ...;
```

```
INSERT INTO employees (id, name, salary)
VALUES (123, 'John Doe', 5000)
ON CONFLICT (id)
DO UPDATE SET
    name = EXCLUDED.name,
    salary = EXCLUDED.salary;
```

Задача #1

1.1 Создание базы данных и таблиц

Задача:
Создайте новую базу данных company_db и две таблицы: employees и departments. Таблица employees должна содержать поля id (целое число, первичный ключ), name (строка), department_id (целое число, внешний ключ на таблицу departments), и salary (число). Таблица departments должна содержать поля id (целое число, первичный ключ) и department_name (строка).

1.2. Вставка данных

Задача:
Заполните таблицы employees и departments несколькими записями. В таблицу departments добавьте три отдела, а в таблицу employees — пять сотрудников, распределённых по созданным отделам.

1.3. Выполнение запросов с использованием DML

Задача:
Напишите SQL-запрос, который выбирает имена всех сотрудников и названия их отделов, используя оператор JOIN.